STAKE - Spatial Temporal Attention Keyframe Extractor using CNNs and CLIP Vision Language Model for Kathakali dance

Ashiq Firoz[1], Navneeth Sreenivasan[2], Nandini J. Warrier[3], and Manu Madhavan[4]

Department of Computer Science and Engineering,
Indian Institute of Information Technology Kottayam
Kerala, India
{ashiq22bcd13, navneethsreenivasan.25phd21010, nandinijw, manum}@iiitkottayam.ac.in

Abstract Kathakali is a dance form from the Indian state of Kerala where different hand gestures and facial expressions can be combined with body positions and angles to represent words used in daily life. Translating the words from videos of Kathakali dance postures requires keyframes consisting of still postures, which in turn requires means of keyframe identification. In this work, we propose a novel approach for keyframe identification and extraction, called STAKE (Spatial Temporal Attention Keyframe Extractor), created using CNN models, attention and LSTMs. We collect videos for twenty words used in Kathakali and manually identify the keyframes from these videos, which serve as Ground Truth. We then use different CNN models such as ResNet50, DenseNet121, EfficientNetB0 and MobileNetV2 to extract features, combine them with the features identified by the CLIP Vision Language model, and extract the temporal dependencies between image frames using Bi-LSTM. The detected keyframes, on comparison with the Ground Truth, gives 77% accuracy, and comparable precision@9, recall@9, Macro F1-score, Weighted F1-score, Mean Absolute Temporal Error (MATE) as well as Dynamic Time Warping (DTW). We further conducted ablation studies as well as compared the performance of our best performing model with the existing baseline models. Further, we also performed Wilcoxon's signed rank test to check if the distribution of the scores for each video differ significantly between the different models. Our future work will concentrate on using the classifying the extracted keyframes to translate the videos of Kathakali dance to natural language words, for use in digital archival as well as heritage preservation.

Keywords: Kathakali · Keyframes · Keyframe Extractor · CNN · CLIP · LSTM

1 Introduction

Kathakali, a classical dance form originating from the southern Indian state of Kerala, is renowned for its intricate body movements and enticing facial ex-

pressions, which serve as a medium to narrate stories drawn primarily from the folklore of Kerala. Central to Kathakali are its expressive elements - *Mudras* and *Navarasas* - that create an enticing experience for the audience. *Mudras* represent the hand gestures shown in Kathakali, whereas *Navarasas* refer to the facial expressions. Mainly, 24 different *Mudras* and 9 different *Navarasas* exist in Kathakali, which can be combined with different body positions and body angles, to represent words used in daily life[1]. Due to this reason, Kathakali is also considered as a sign language. Short video clips of Kathakali performances are usually used to depict a word represented by the postures in that video clip.

The dance postures in Kathakali can be classified into still postures and transition postures. Still postures are the postures where there is no movement, whereas transition postures have some form of movement. Translating the words from videos of Kathakali dance postures requires understanding the still postures, which brings about the need for keyframes. Every video consists of image frames, which can be classified into Keyframes and Transition frames. Keyframes represent the key (still) postures in a video, whereas transition frames are used to represent the transition postures. Kathakali dance consists of keyframes and transition frames which come one after the other.

Generally, the transition frames are blurry and do not contribute a lot in learning, and therefore, a sequence consisting of only keyframes can be used for translation. Using keyframes rather than all the frames can also help in significantly reducing the computational load as well as training time, which leads to an increase in efficiency. Apart from sign language translation, keyframe identification is also used in other domains such as surveillance, cancer detection, polyp identification etc. [2] [3] [4]. Furthermore, Keyframe analysis can help in various tasks such as Video Summarization or Video Classification [5] [6]. Due to these reasons, keyframe identification and extraction is an area where a lot of research is going on.

In this work, a novel keyframe extraction model called STAKE (Spatial Temporal Attention Keyframe Extractor), which combines CNN models, Vision language models as well as LSTMs, is proposed. We first collect videos of twenty words used in Kathakali and manually identify the keyframes from them, which serve as Ground Truth. We then use different CNN models such as ResNet50, DenseNet121, EfficientNetB0 and MobileNetV2 to collect features of each image frame. We also use the CLIP Vision Language model to extract features, from the image frames. The features identified by the CNN models as well as CLIP are combined together and a Bidirectional LSTM is used to identify the temporal information between the features. We then classify the frames into keyframes and transition frames. Accuracy, Precision@9, Recall@9, Macro F1-score, Weighted F1-score, Mean Absolute Temporal Error (MATE) and Dynamic Time Warping (DTW) are used to evaluate the classification results as well as the temporal alignment between the Ground Truth keyframes and predicted keyframes.

The proposed system is designed as the first stage of our larger pipeline for translating Kathakali dance videos into the words they represent. By automatically identifying keyframes and removing transition frames, our method signif-

icantly reduces noise and can help in efficient word translation in future. This can also help in dance video generation in future, by identifying the keyframes and generating movements in between the keyframes, to simulate dance. Beyond this, the extracted keyframes can also facilitate the digital storage and archival of Kathakali performances by identifying key dance postures, ultimately leading to the preservation of Kathakali dance form. They can also help further dance education by helping students focus on the important postures, *Mudras*, and *Navarasas* during training. Furthermore, this method can be applied to video summarization, indexing and retrieval of dance sequences, as well as the development of frameworks for studying and analysing Indian Classical Dance (ICD).

The rest of this paper is organized as follows. Section II comprises the Literature Survey which reports the previous methods used to perform Keyframe extraction; Section III presents the Methodology of our work in detail; Section IV reports the Experiments we ran and the results we got; and Section V concludes the work.

2 Literature Review

Previous works focusing on sign language translation have mainly used various kinds of deep learning models and image processing techniques. Donepudi, Swapna, et al. [7] used a system comprising of YOLOv11 and LSTMs to detect the hand postures using wearable gloves and identify temporal information across the videos. Ingoley, Shilpa, and Jagdish Bakal. [8] used VGG16 and MobileNetV2 rather than LSTMs to translate Indian Sign Language postures representing numbers from zero to nine, as well as thousand, lakh (hundred thousand) and crore (ten million). Renjith, S., Manazhy Rashmi, and Sumi Suresh [9] created an architecture, termed as Time-distributed CNNs, rather than LSTMs for translation of Chinese Sign Language (CSL) and Arabic Sign language (ArSL). However, all these works have used all the frames, rather than just focusing on the keyframes, leading to longer computational time and a larger computational load, which in turn leads to a reduction in efficiency.

However, some previous works have focused on keyframe identification and extraction. Most of them have focused on deep learning and computer vision based techniques, some have used clustering-based approaches and a few have used other approaches. Some of them are mentioned below.

2.1 Deep Learning and Computer Vision based approaches

Huang, Ruobing, et al. [3] manually annotated breast cancer images to serve as Ground Truth, and developed a reinforcement learning model to compare the model output with the Ground Truth images, for use in classification. A Bi-LSTM model was used to classify the videos to detect cancer. Prakash, M. Madhura, and G. N. Krishnamurthy [4] developed a model combining MobileNetV2, UNet and attention from Transformer architecture to get the depth

estimation from image frames of polyp in human intestines, and using a combination of information entropy, key points and image moment from these depth estimated image frames, identified the key frames. Mo, Clinton, et al. [10] created skeletal images of people from the images, use it as a graph and find similar skeletal images using a reinforcement-learning model which rewards or penalises the model. Zarif, Sameh, et al. [11] first removed the blurry frames using a combination of intensity, contrast, and motion detection between pixels. They then extracted the low-level features using ConvNext architecture and clustered these features using a combination of Bayesian networks, PCA and K-Medoids clustering, to get the keyframes. Zheng, Bowen, et al [12] derive control signals from the root trajectory paths of humans and developed equations to measure the difference between consecutive frames. They create a diffusion model to automatically generate keyframes and further create a synthetic dataset to test their model.

Tan, Guanghua, and Jinxi Zou [13] used an approach by calculating the colour histogram and similarity matrix of each frame, classified the videos to smaller video shots and reported the centre frame of each shot was as the keyframe. Kavitha, J., et al. [14] first removed the blurry frames manually, then applied a three-stage method to detect keyframes. They first applied a Multi-Criteria Decision Making system based on visual quality and motion details, then performed low-level feature extraction, normalization, weight assigning, and score calculation on top of the system, and finally, applied a RNN model with LSTM to extract the keyframes. Ashiquzzaman, Akm, et al. [15] used a combination of Bhattacharyya distance as well as Histogram of images to develop equations and created a threshold to detect keyframes, then used it to classify surveillance videos with fire and smoke. Singh, Pratibha, and Alok Kumar Singh Kushwaha [16] extracted all the frames, used a sliding window to filter out frames, and then used SURF, GIST and DNS to extract the keyframes. They measured the temporal distance between simultaneous frames using a threshold value.

2.2 Clustering-based approaches

Tan, Kailong, et al. [4] used an adaptive clustering based method similar to K-means, by initially creating random clusters, found the cluster centres, and detected keyframes from cluster centres using distance-based metrics. Tang, Hao, et al. [5] focused on extracting features in frames using ResNet, calculated the distance between the features, and used an algorithm to create clusters from the features. They extract the top 1-2% neighbours of the cluster centres as the keyframes and further perform video classification using LSTMs on the keyframes. Lv, Chaohui, and Yiyang Huang [17] removed the blurry frames using motion blur detection and used nearest neighbour clustering algorithm to find keyframes from the cluster based on similarity between the frames. Xiao, Yang Dong, Cheng Xin Min, and Ran Mao Ke [18] improved the speed of clustering by adding a lightweight-SAM model. The lightweight-SAM model is able to identify objects in the image, use these identified objects to identify the cluster

centres, and use the centres for keyframe identification. Ai, Yu, Ce Xu, and Yansong Bai [19] encoded the frame bit number of each frame and then found the motion vector and coding unit depths of each frame bit. These are used to create clusters and the frame closest to the cluster centre is identified as the keyframe.

2.3 Other approaches

Rajan, Manjusha, and Latha Parameswaran [2] create a method to classify keyframes based on Genetic Algorithm. They identify the keyframe as the fittest chromosome and perform crossover and mutation across all the frames to identify the keyframes. Navyasri, M., and G. Jaya Suma [20] use cosine similarity and structural similarity between the neighbouring frames to detect keyframes. They also create a deep learning model with channel attention to classify the keyframes of sign language videos. Bhuyan, Himadri, et al. [21] first perform uniform sampling across Bharatanatyam dance videos to reduce the number of images, then classify a frame based on three different classification techniques - threshold value, majority voting mechanism and using Machine learning, specifically, SVM and CNN. Manually annotated images are used as the Ground Truth for comparison with the output of machine learning approach. Peer Mohamed Appa, M. A. Y., et al. [22] have used a statistical feature extraction technique, by measuring the mean and standard deviation between consecutive images and found the similarity between consecutive frames. The frames with more distance are classified as keyframes and it is further used to identify abnormal vehicles from surveillance videos. Qiao, Shunli, and Honglin Liu. [23] identified keyframes by splitting large surveillance videos into smaller video shots, and extracting the first and last frame from these shots. These frames from the shots are combined to get the keyframes for a large video, and these are then passed through a modified Inception architecture to learn temporal dependencies to classify criminal activities.

2.4 Limitations of the existing approaches

The above-mentioned authors have used a wide variety of approaches for keyframe extraction. However, a few limitations exist across the existing approaches. The outputs of almost all the previous works, including clustering-based approaches, computer vision-based approaches, uniform sampling etc. consists of transition frames, thereby leading to inaccuracy. Specialised data such as sign language or dance data may need object detection algorithms, and using just CNNs and LSTMs could lead to lower accuracy. Reinforcement learning requires the use of designing effective reward and penalty functions, however they may be specific to different applications.

To overcome the limitations of the existing approaches, we propose a novel keyframe identification and extraction method, called STAKE. Rather than using CNN or LSTMs by themselves, we combine CNNs and LSTMs with CLIP Vision Language model to detect the objects in a video. Using a combination of

the features identified from the CNN models as well as the CLIP model, along with the temporal information learnt by the LSTMs will help in the classifying frames into keyframes and transition frames.

3 Methodology

We first collected the dataset, which consists of short video clips (5-10 seconds long) of Kathakali dance, each representing a word. Next task was to extract all the frames from each of the videos following which, we manually identified the keyframes. These manually identified keyframes are considered our Ground Truth values and will be used to compare with the output of our model. After this, the image frames are passed to the STAKE (Spatial Temporal Attention Keyframe Extractor) model. These frames are passed through the CNN models as well as CLIP Vision Language Model to extract the features. The outputs of the CNN as well as the CLIP are concatenated together and are then passed through a Bi-LSTM model to identify the temporal dependencies. Finally, these frames are classified into keyframes and transition frames and metrics are calculated by comparing the output keyframes given by the model with the manually identified keyframes (Ground Truth). Accuracy, Precision@9, Recall@9, Macro F1-score, Weighted F1-score, Mean Absolute Temporal Error (MATE) and Dynamic Time Warping (DTW) are used to evaluate the classification results as well as the temporal alignment between the Ground Truth keyframes and identified keyframes.

3.1 Dataset collection

We used twenty Malayalam words commonly expressed in Kathakali. These words, written in Malayalam , along with their English transliterations, and English meanings are provided in Table 1. Ambiguities are present while representing a word in Kathakali language. One such example is the word, *Nee* (English meaning "You"), which has two different types of representations, using two different Navarasas (facial expressions), *Shringaaram* and *Roudram*. We collected these separately, and this brought the total to about twenty one different representations from the twenty words. We recorded the video for each representation ten times with the help of an expert in Kathakali dance, with small changes in the body angles and camera movement. The data was collected in 1080p as well as 4K resolutions with some videos recorded in 30 fps and some in 60 fps. All the videos were recorded in an iPhone 17. These settings helped us ensure that the videos emulated real-world data with different resolutions, varying speed as well as from different angles. The videos were named according to the word they represented and the number of times they were performed. For example, if the word being represented was *Aana* (Elephant) and the second iteration of this word was being recorded, that video was named as *Aana_2*.

Malayalam words	**English transliterations**	**English meanings**
ആകാശം	*Aakasham*	Sky
ആന	*Aana*	Elephant
അച്ഛൻ	*Acchan*	Father
ഭൂമി	*Bhoomi*	Earth
ബ്രഹ്മാവ്	*Brahmaavu*	God Brahma in Hindu mythology
ചന്ദ്രൻ	*Chandran*	Moon
ഇരിക്കുക	*Irikkuka*	To sit
കാട്	*Kaadu*	Forest
നടക്കുക	*Nadakkuka*	To walk
നീ	*Nee*	You
പാദം	*Paadam*	Feet
സമുദ്രം	*Samudram*	Ocean
സത്യം	*Satyam*	Truth
ശിവൻ	*Shivan*	God Siva in Hindu mythology
സിംഹം	*Simham*	Lion
സൂര്യൻ	*Sooryan*	Sun
സ്വർഗം	*Svargam*	Heaven
വീട്	*Veedu*	House
വിഷ്ണു	*Vishnu*	God Vishnu in Hindu mythology
വൃക്ഷം	*Vriksham*	Tree

Table 1. Words used

3.2 Manual keyframe identification

Our next task was to extract all the frames from the videos and name them. We followed the naming convention <word>_<iteration>_<frame_number> to name the frames. For instance, if the video currently being considered was *Aana_1*, then the frames were named as *Aana_1_001*, *Aana_1_002*, etc. Our next task was to manually identify the keyframes from all the frames in all the videos. We found that most of the words have three still positions - in the starting, middle, and the end. Therefore, we decided to select three sets of keyframes corresponding to these three still positions from each video to ensure consistency. For instance, if the three still positions from the video *Aana_1* correspond to the frame numbers 1-10, 99-104, and 185-205, then the frames *Aana_1_001* to *Aana_1_010*, *Aana_1_099* to *Aana_1_104* and *Aana_1_185* to *Aana_1_205* are extracted. These manually extracted keyframes are used as the Ground Truth values. Some of the manually identified keyframes of the video *Aana_5* are shown in Figure 1. Details on the number of frames across every video for each word is shown in Table 2.

From Table 2, we can see that the number of frames across the videos is not uniform, since the videos vary in length and the number of frames per second (fps) is also different. The video *Aakasham_1* consists of 415 frames while the video *Aakasham_2* consists of only 115 frames. Difference in duration exists across videos with videos of the same word itself ranging from 5 seconds to 10

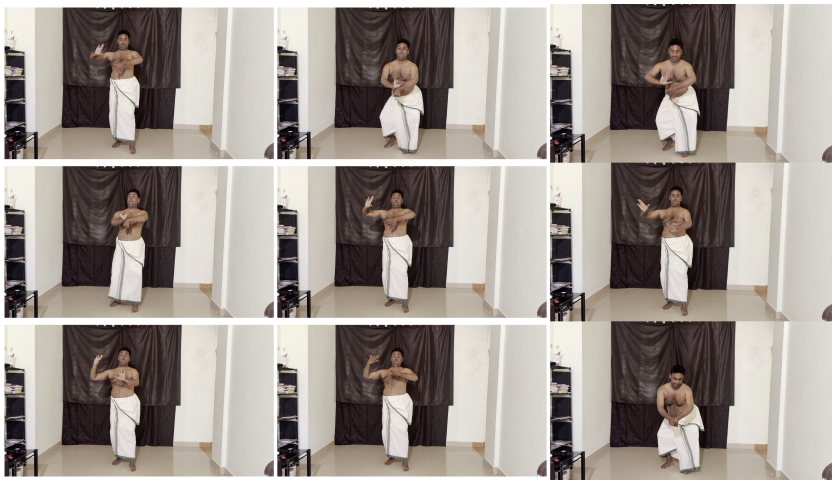


Figure 1. Examples of manually identified keyframes

seconds. Videos numbered 1 to 4 are all taken at 30 fps and therefore have lesser number of frames, as compared to videos numbered 5 to 10, taken at 60 fps. However, these conditions ensure that our video dataset collection emulates real-life data with varying lengths and differing number of frames.

The list of manually identified keyframe ranges for every video for the word *Aakasham* is given in Table 3. From Table 2, we can see that *Aakasham_1* contains 415 frames. On comparison with Table 3, we can see that out of these 415 frames, only the frame ranges 1-24, 81-112, and 173-199 correspond to keyframes. This corresponds to roughly 83 keyframes out of 415 frames, making the rest transition frames. A similar trend can be observed across the other videos as well. This shows us that there is a huge class imbalance across the videos, with transition frames dominating the keyframes across every video. However, this trend also shows us that even if the number of frames vary by video, the keyframes exist in the starting, middle and end of every video.

3.3 STAKE Model Architecture

Our STAKE model consists of four components - the CNN model, the CLIP Vision Language model, the Bi-LSTM model, and the Classifier. The detailed STAKE model architecture is shown in Figure 2. All the frames of every video are trained by the CNN model and the CLIP Vision Language Model in parallel, then the features identified by both the CNN model and CLIP are combined together. These image frames are then trained by Bi-LSTM to extract the temporal dependencies between the frames. Finally, they are classified into keyframes and transition frames. Since our videos consist of both general features such as shapes and edges, as well as Kathakali-specific semantic features such as *Mudras* and *Navarasas*, using only a CNN model or CLIP model will prove insufficient.

Word	**Video Number**									
	1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
Aakasham	415	115	499	405	288	189	431	583	514	522
Aana	205	200	278	422	228	224	528	516	389	544
Acchan	239	119	246	409	325	186	453	380	394	464
Bhoomi	265	248	527	556	284	273	477	519	581	516
Bhrahmaavu	248	188	169	310	363	189	259	304	280	357
Chandran	371	176	399	506	205	187	426	496	547	461
Irikkuka	184	164	376	458	158	137	381	334	286	309
Kaadu	362	256	634	561	314	219	641	574	545	799
Nadakkuka	313	292	656	544	315	315	512	833	740	564
Nee_Roudram	130	175	533	538	205	236	424	551	563	511
Nee_Shringaaram	244	330	985	662	309	354	810	773	937	794
Paadam	215	178	431	374	209	199	448	314	568	376
Samudram	409	255	677	520	344	316	635	761	740	770
Satyam	241	125	336	319	195	199	445	411	406	445
Shivan	163	135	272	319	245	110	216	266	249	210
Simham	190	97	221	239	87	127	271	282	221	263
Sooryan	265	126	361	324	108	136	359	445	466	397
Svargam	250	216	506	604	272	266	465	796	514	590
Veedu	162	95	218	341	210	149	336	261	305	234
Vishnu	209	180	132	292	242	190	319	296	336	322
Vriksham	153	124	225	210	199	169	559	545	496	482

Table 2. Number of frames across videos for each word

Therefore, we decided to use a combination of the two for feature extraction. The different CNN models we used are ResNet50, DenseNet121, EfficientNetB0 and MobileNetV2.

Feature extraction using CNNs All the image frames are trained separately by the various CNN models such as ResNet50, DenseNet121, EfficientNetB0 and MobileNetV2, which helps in extracting the spatial features from the image frames. ResNet50 [24] is a widely used benchmark CNN architecture, consisting of skip connections, which helps gradients flow through the network. It is widely used in general image-related tasks and has around 25.6 Million parameters. DenseNet121 [25] is a CNN model with dense layers, where each neuron in a layer is connected to each neuron in the preceding and subsequent layers. It helps in reusing features through the dense layers, and has a size of around 8 Million parameters, making it less heavy than ResNet50. EfficientNetB0[26] is a lightweight CNN model which consists of Squeeze-and-Excitation blocks and inverted residual blocks. It uses compound scaling to uniformly scale depth, width and resolution optimally, and has 5.3 million parameters. MobileNetV2 [27] is a lightweight CNN model which utilizes depth-wise separable convolutions and inverted residuals. It has 3.5 Million parameters, making it a powerful model across edge devices.

Video	Range 1	Range 2	Range 3
Aakasham_1	1-24	81-112	173-199
Aakasham_2	1-8	9-16	48-61
Aakasham_3	1-9	158-185	321-346
Aakasham_4	1-10	52-96	253-333
Aakasham_5	1-14	94-111	179-203
Aakasham_6	1-10	19-35	109-120
Aakasham_7	1-19	103-140	272-311
Aakasham_8	49-70	180-207	349-394
Aakasham_9	1-12	120-139	246-315
Aakasham_10	28-63	191-211	323-396

Table 3. Number of frames across videos for word *Aakasham*

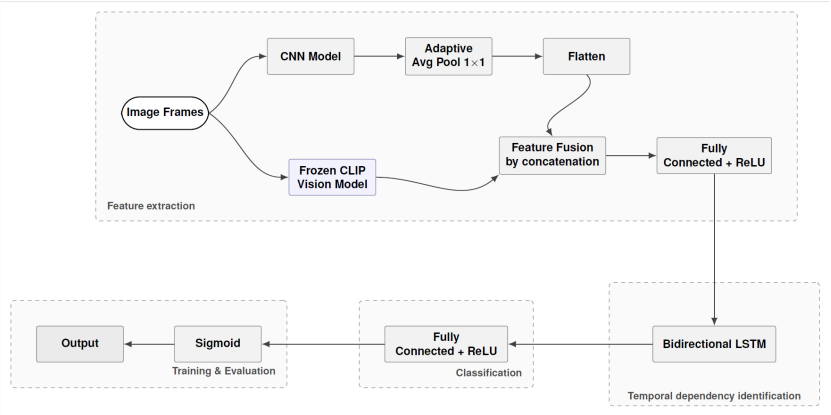


Figure 2. Model Architecture

These models are diverse with differing number of parameters, diverse architecture, and different use cases. ResNet50, though heavy on parameters, is a benchmark model and is found in almost all image-related tasks. DenseNet121 has a very complex architecture with a large number of dense layers, but this also makes it efficient for parameter reuse. EfficientNetB0 and MobileNetV2 are both lightweight models, and therefore, can be deployed in edge devices. Due to these factors, these specific CNN models are used for feature extraction.

Each CNN model gives outputs of different sizes, which could lead to inconsistency in subsequent layers. To mitigate this issue, we added an Adaptive Average Pooling layer of size 1x1 after the outputs of each CNN. These were then flattened to help in fusing with the outputs of the CLIP model.

Feature extraction using CLIP Vision Language Model Large language Models, also known as LLMs, are very large deep learning models that are pre-trained on vast amounts of text data [28]. The underlying transformer is a set of neural networks that consist of an encoder and a decoder with self-attention

capabilities. The encoder and decoder extract meanings from a sequence of text and understand the relationships between words and phrases in it. The self-attention mechanism helps to learn the importance of one word with another, with multi-head attention doing it in parallel to capture various types of relationships. A Vision Language Model (VLM) is an AI system built for emulating LLMs in computer vision-related tasks [29]. VLMs can process and provide advanced understanding of video, image, and text inputs to generate text responses. CLIP (Contrastive Language-Image Pre-training) [30] is a multimodal VLM which is widely used in image segmentation tasks. Unlike other Vision Language Models, CLIP was trained on 400 million image-text pairs and can perform classification on datasets it was not trained on, helping it to easily capture high-level features, such as the relationships between images and text [31].

In our work, we use the pretrained CLIP base model, which consists of twelve layers. Instead of training the CLIP from scratch, we froze all the layers of the CLIP model and directly trained our images with CLIP. This ensures that CLIP does not overfit on our dataset, while also reducing the computational load.

Fusing the features Our next task was to fuse the features identified by the CNN models with the features identified by the CLIP model. The features identified by CLIP are concatenated with the features identified by CLIP. CLIP gives an output of 768, but the outputs given by the CNN model could vary depending on which specific CNN model is being used. Due to this reason, we added a Fully Connected layer of size 1024 followed by a ReLU layer, after the feature concatenation. These are listed below -

1. Fully connected layer - The fused features are then connected to a fully connected layer of size 512 neurons. Each neuron in the previous layer is connected to each neuron in the next layer. The size, 512 neurons, was chosen to correspond to the size of the Bi-LSTM model
2. ReLU Activation - Activation functions are used to bring non-linearity to the model, and ReLU is one of the most common and most powerful activation functions. The equation of ReLU is shown below:

$$f(x) = \max(0, x) \tag{1}$$

Addition of Temporal information using Bi-LSTM After the feature concatenation, a Bi-LSTM model is used to train the image frames to ensure that the temporal dependencies between the features are learnt by the model. A Bidirectional LSTM (Bi-LSTM) is used to manage the vanishing gradient problem faced by RNNs and capture temporal dependencies in sequential data by monitoring information flow in forward as well as backward direction simultaneously [32]. Though Bi-LSTM is more heavier than regular LSTM, it has more accuracy and consists of more connections, which could potentially help learn more information. Our Bi-LSTM consists of two layers, each consisting of 512 hidden units.

Classification Head We now add the following layers -

1. Fully connected layer - A Fully connected layer of size 1024 is added. This ensures that both the layers of LSTM, each containing 512 neurons is flattened and each neuron from each layer of Bi-LSTM is connected to each neuron in the fully connected layer
2. ReLU - A ReLU Activation function is added to bring non-linearity to the model

4 Experiments and Results

4.1 Experimental setup

We used 70:10:20 train-validation-test split to evaluate our model and ran it for 5 epochs. We experimented on the number of epochs and found that the models started to converge by epoch 5. The model training and testing was done using Google Colab's T4 GPU in Python. The batch size was set to 1 due to computational constraints. We chose a cut-off value of 5 for the Bi-LSTM, thereby making the Bi-LSTM store information of 5 frames at a given time, leading to lesser computational overhead. The model is trained to predict either 0 (transition frame) or 1 (keyframe). A learning rate of 0.0001 along with Adam optimizer was used.

We adopted a fixed train-validation-test split rather than cross-validation because the dataset consists of video sequences. Our train-validation-test split was fixed in such a way that the first seven videos of every word are used for training, eight video is used for validation, and the ninth and tenth videos are used for testing. Maintaining video-level independence between training and testing is essential to prevent data leakage arising within frames from the same video. Furthermore, deep learning models require substantial computational resources, making repeated retraining across multiple folds impractical. A standalone test set was therefore used for final evaluation.

Due to the effect of class imbalance, we experimented with sampling strategies, specifically oversampling and undersampling strategies. Oversampling was performed by increasing the number of minority frames to equal that of the majority frames, however, oversampling increased the number of similar keyframe samples within the training set, thereby resulting in severe overfitting. We performed undersampling by decreasing the number of majority frames to equal that of the minority frames, but this substantially reduced the number of training frames, leading to underfitting and degraded model performance. Other techniques such as generating artificial keyframes were considered to increase the size of training data without duplication, but since these methods require significant computational resources, we decided that this method was impractical as well. Based on these observations, we decided to retain the original data distribution.

We used Sigmoid activation function to produce output probabilities and Binary Cross Entropy (BCE) to compute the loss. The model trains by learning from the Ground Truth keyframes and tries to perform better across the epochs.

The loss values are backpropogated and the weights and biases are adjusted by the model. The equation of sigmoid activation is shown below:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{2}$$

The accuracy, Macro F1-score, and Weighted F1-score, are calculated by comparing the output of the model with the manually identified keyframes (Ground Truth). To ensure consistency in accuracy, we select nine frames which the model detected as keyframes, having the highest confidence score predicted by the sigmoid activation function at the output, per video. This ensures that the number of keyframes we consider from each video are equal, and videos with higher number of identified keyframes do not dominate the evaluation. We experimented with other values as well, such as three, or thirty keyframes across videos. Using three keyframes per video did not give us enough keyframes to compare, whereas choosing thirty keyframes per video lead to transition frames also appearing from videos, leading to wrong predictions. However, since using only nine keyframes per video could introduce bias in our imbalanced dataset, we decided to also use other metrics such as Mean Absolute Temporal Error (MATE), Dynamic Time Warping (DTW), Precision@9, and Recall@9. Mean Absolute Temporal Error (MATE) measures difference between the timestamps between the predicted and Ground Truth keyframes. Dynamic Time Warping (DTW) measures the minimum total distance required to align the predicted keyframe sequence and Ground Truth keyframe sequence. For Accuracy, Macro F1-score, Weighted F1-score, Precision@9, and Recall@9, higher values indicate better performance, whereas for DTW and MATE, lower values indicate better performance.

4.2 Results

The Accuracy, Macro F1-score, Weighted F1-score, Mean Absolute Temporal Error (MATE), Dynamic Time Warping (DTW) distance, Precision@9 and Recall@9, from each of the four sets of models is shown in Table 4. The predicted image frames for the word *Aana* (Elephant) from the video *Aana_1* are shown in Figure 3.

Model	Accuracy	Macro F1-Score	Weighted F1-Score	MATE ↓	DTW ↓	Precision@9	Recall@9
ResNet50+CLIP+Bi-LSTM	**0.7730**	**0.3855**	**0.7814**	**3.0503**	**34.6429**	**0.3519**	**0.4366**
DenseNet121+CLIP+Bi-LSTM	0.7645	0.3623	0.7732	2.7884	33.5952	0.3307	0.4119
EfficientNetB0+CLIP+Bi-LSTM	0.7677	0.3710	0.7763	2.7407	33.3095	0.3386	0.4180
MobileNetV2+CLIP+Bi-LSTM	0.7687	0.3739	0.7773	3.1455	35.5000	0.3413	0.4268

Table 4. Results of evaluation (↓ indicates lower is better)

All the four sets of model combinations show relatively same values across the metrics, despite utilizing CNN models with varying architecture, number

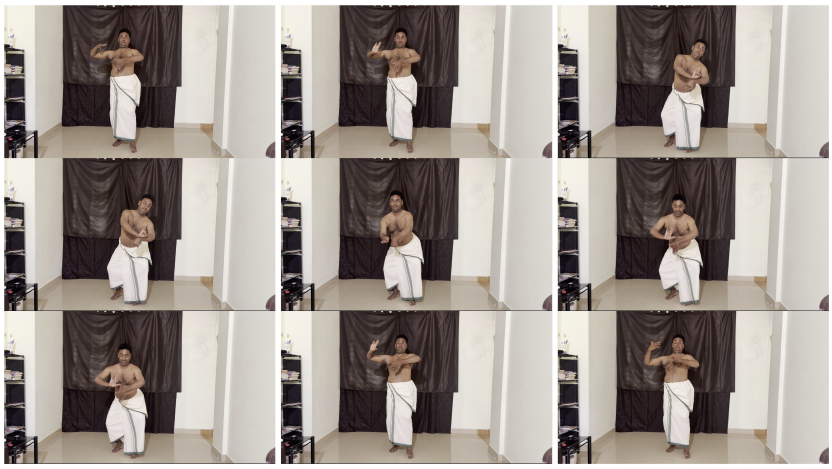


Figure 3. Predicted keyframes

of parameters and size. This shows that the CNN model's contribution is secondary to that of the CLIP model and the Bi-LSTM temporal modelling. The best performing model was ResNet50+CLIP+Bi-LSTM, with the highest values from Accuracy, Macro F1-Score, Weighted F1-Score, Precision@9 and Recall@9. However, it does not have a good performance when it comes to MATE and DTW, with the models involving DenseNet121 and EfficientNetB0 performing better than the one with ResNet50. Finally, ResNet50+CLIP+Bi-LSTM was selected as the best model because it consistently outperformed the other models on the primary keyframe identification metrics (Accuracy, Precision@9, Recall@9, Macro F1-Score ad Weighted F1-score), and it's performance still remains competitive across MATE and DTW.

Another observation we can make is that the values of metrics of lightweight models like MobileNetV2 (with 3.5M parameters), EfficientNetB0 (with 5.3M parameters) and DenseNet121 (with 8M parameters) have almost the same value compared to that of heavy architectures such as ResNet50 (with 23M parameters). We can see that that increasing the number of parameters do not show a better accuracy in the proposed architecture, which leads us to conclude that lightweight CNN models are more suitable than heavier models for keyframe extraction, especially on edge devices. However, ResNet50 is a widely used benchmark model is found across almost all image-related tasks. Therefore, even though it has comparable accuracy with the other models and has a heavier architecture, usage of ResNet50 is significant for comparison.

A third observation we can make is that the accuracy is higher while the Precision@9, Recall@9 and Macro F1-score are lower. This can be explained by the fact that our video has a significantly higher number of transition frames (1) as compared to keyframes (0). Since the keyframes are under-represented, our model trains on far more transition frames than keyframes, which could lead

to a lower Macro F1-score, as evidenced by our results. To mitigate this issue, Weighted F1-score was also used, since it gives more weight to the classes with lower number of frames, and therefore, can be considered as a better metric for handling classes with data imbalance.

MATE value being 3.0503 for the ResNet50+CLIP+Bi-LSTM combination means that on average, the predicted keyframes are three keyframes away from the corresponding Ground Truth keyframes. DTW measures the total distance for matching the Ground Truth keyframes with the predicted keyframes, accordingly, a value of 34.6429 could mean that the total distance between Ground Truth and predicted keyframes is more, since it could become smaller or larger according to video lengths.

4.3 Comparison with existing baseline models

We also evaluated the performance of our model with existing baseline models from the literature review. We selected our top performing model, ResNet50+CLIP+Bi-LSTM, and compared it with existing models, with the same dataset and the same experimental setup. The results are summarised in Table 5.

Model	Accuracy	Macro F1-Score	Weighted F1-Score	MATE↓	DTW↓	Precision@9	Recall@9
Reinforcement learning + DR-DSN [10]	0.4844	0.3072	0.3627	8.72	18.78	0.5016	0.2357
K-medoids clustering [11]	0.5124	0.3448	0.3984	8.00	9.25	0.5622	0.2625
CNN [15]	0.5326	0.3719	0.4232	6.27	24.16	0.6060	0.2815
Nearest neighbour clustering [17]	0.4968	0.3239	0.3787	8.34	21.22	0.5252	0.2460
Cosine + SSIM [20]	0.5793	0.4346	0.4776	6.92	22.92	0.7070	0.3258
Uniform threshold + Majority voting + SVM and CNN [21]	0.5746	0.4284	0.4724	7.06	24.65	0.6969	0.3205

Table 5. Results of comparison with baseline methods

From the results we can see that our model leads in terms of all the metrics, except DTW. This could be explained due to the fact that individual matched keyframes are accurate between the Ground Truth and predictions, but the overall sequence of frames contain extra frames. This is true in our case, since we have a range of keyframes in our Ground Truth and comparing the predictions with these may not always result in the same number, but could rather also move up or down, depending on which keyframe was given as output.

4.4 Ablation Studies

Our next task was to perform Ablation studies on our model. We ran four different types of ablations - CNN model alone, CLIP model alone, CNN with Bi-LSTM (without CLIP) and CLIP with Bi-LSTM (without CNN). The results are summarised in Table 6.

Model	Accuracy	Macro F1-Score	Weighted F1-Score	MATE↓	DTW↓	Precision@9	Recall@9
Only ResNet50	0.7003	0.2412	0.7130	3.3524	37.8810	0.2169	0.2821
Only DenseNet121	0.7224	0.2971	0.7341	2.7831	43.6667	0.2672	0.3407
Only EfficientNetB0	0.7294	0.3147	0.7408	2.5714	36.2143	0.2831	0.3634
Only MobileNetV2	0.7189	0.2882	0.7308	2.8915	36.3095	0.2593	0.3319
Only CLIP	0.7050	0.2275	0.7174	3.2063	51.7619	0.2275	0.2917
ResNet50 + Bi-LSTM	0.7335	0.3652	0.7742	3.0608	34.4524	0.3333	0.4104
DenseNet121 + Bi-LSTM	0.7445	0.3652	0.7742	2.8042	35.8571	0.3333	0.4139
EfficientNetB0 + Bi-LSTM	0.7405	0.3594	0.7721	2.7143	33.6190	0.3280	0.4004
MobileNetV2 + Bi-LSTM	0.7484	0.3188	0.7577	3.3571	36.9286	0.2910	0.3580
CLIP + Bi-LSTM	0.7596	0.3912	0.7697	2.5185	39.7143	0.3519	0.4449

Table 6. Results of Ablation studies

From the ablation study, it is clear that our although a few models do give a solid performance, our combination of Resnet50+CLIP+Bi-LSTM still stands the strongest. The ResNet50+CLIP+Bi-LSTM model tops the performance across almost all the metrics, except for MATE. Although the accuracy of CLIP alone is comparable to the accuracy of our model, the values of the other metrics of CLIP alone is sufficient to understand that our model performance is still the highest. We can see that addition of Bi-LSTM increases the the performance across almost all the metrics. We can also conclude that the performance of the models become better when combined, rather than alone. CNN models do not work well by themselves and neither does CLIP , though combining them with Bi-LSTM does give better accuracy. This again shows us that our combination of RessNet50+CLIP+Bi-LSTM stands out the strongest.

4.5 Statistical Significance tests

We further ran statistical significance tests across the baseline models as well as the ablation models by comparing them with our best performing model. We used Wilcoxon's signed rank test to evaluate per-video metrics of our best performing model with the ablations as well as with the baseline models. The tests were performed with our ResNet50+CLIP+Bi-LSTM compared with the ablations as well as with the baselines. Five metrics which could be computed per video were used, namely F1-score, Dynamic Time Warping (DTW), Mean Absolute Temporal Error (MATE), Precision@9 and Recall@9. The results show that our model shows a good performance in DTW, F1-score, Precision@9, and Recall@9, but does not have a good performance in MATE, which concurs with our previous findings. It achieves significantly lower DTW than CLIP alone, CNN [15], Reinforcement learning + DR-DSN [10], and Uniform threshold +Majority Voting + SVM and CNN [21], demonstrating substantially better overall temporal alignment. It also achieves significantly higher F1-score and Precision@9 than ResNet50, CLIP, DenseNet121, and MobileNetV2, and significantly higher Recall@9 than ResNet50, CLIP, DenseNet121, MobileNetV2, and Reinforcement learning + DR-DSN [10]. In most of the cases, the rank-biserial correlation value favoured our model significantly with almost all comparisons having an alignment towards our model and a larger magnitude.

Our model consistently provides better performance across complementary evaluation criteria, especially sequence-level alignment (DTW), F1-score, Precision@9, and Recall@9, while certain baselines and ablations perform exceptionally well on metrics such as MATE. However, according to the general pattern, our model provides the overall best balance between precise keyframe retrieval and temporal sequencing.

5 Conclusion and Future Work

Keyframe identification is one of the most important works used in various video-related tasks, such as video summarization or classification. In this work, we present a novel keyframe identification and extraction method using CNN models, VLMs and LSTMs. CNN Models such as ResNet50, MobileNetV2, EfficientNetB0 and DenseNet121 as well as CLIP Vision Language model is used to identify the features in Kathakali. Bi-LSTM is used to extract the temporal dependencies between the frames and a classifier is used for frame classification. Both classification as well as temporal metrics were used to calculate the performance of our model. Comparison with several baseline models were done and ablation studies were conducted. Furthermore, Wilcoxon's signed rank test was also conducted to evaluate per-video metrics across the baselines, ablations and our model.

This method presents a novel approach using a combination of CNN models, VLMs and Bi-LSTMs and has a wide variety of applications in domains such as video summarization, indexing and retrieval of videos, dance video generation and classification, etc. We are also planning to design a larger pipeline for translating Kathakali videos into the natural language words they represent using classification methods across keyframes. However, we are yet to explore which features are identified by CNN models and which ones by CLIP, which is a work we are planning to do in the future. Another limitation is that our dataset is small with only 210 videos. We are in the process of building a bigger dataset with more words as well as with more videos, taken from different people, to expand our evaluation. We are planning to publish this expanded work as well as the bigger dataset in the future. Additionally, in future, this keyframe extraction method can be expanded to other sign languages such as Indian Sign Language (ISL), American Sign Language (ASL) etc. to enable ease of translation of sign languages.

Acknowledgements

Nandini J. Warrier and Manu Madhavan received funding sanction from the DST-SHRI Scheme, Government of India (GrantNo. DST /SHRI-14/2023 (C)) to support this research.

References

1. Venu, Gopalan. (2000). "The language of Kathakali: (Notations of 874 hand gestures)". Kairali
2. Rajan, Manjusha, and Latha Parameswaran. "Key frame extraction algorithm for surveillance videos using an evolutionary approach." Scientific Reports 15.1 (2025): 536.
3. Huang, Ruobing, et al. "Extracting keyframes of breast ultrasound video using deep reinforcement learning." Medical image analysis 80 (2022): 102490.
4. Prakash, M. Madhura, and G. N. Krishnamurthy. "An innovative transfer learning-polyp detection from wireless capsule endoscopy videos with optimal key frame selection and depth estimation." Biomedical Signal Processing and Control 108 (2025): 107963.
5. Tan, Kailong, et al. "Large model based sequential keyframe extraction for video summarization." Proceedings of the International Conference on Computing, Machine Learning and Data Science. 2024.
6. Tang, Hao, et al. "Deep unsupervised key frame extraction for efficient video classification." ACM Transactions on Multimedia Computing, Communications and Applications 19.3 (2023): 1-17.
7. Donepudi, Swapna, et al. "A Sign Language Recognition System for Understanding Actions of Speech-Impaired-Individuals." 2025 7th International Conference on Intelligent Sustainable Systems (ICISS). IEEE, 2025.
8. Ingoley, Shilpa, and Jagdish Bakal. "Interpretation of Indian sign language to text and speech to communicate with speech and hearing impaired community." Procedia Computer Science 258 (2025): 1980-1992.
9. Renjith, S., Manazhy Rashmi, and Sumi Suresh. "Sign language recognition by using spatio-temporal features." Procedia Computer Science 233 (2024): 353-362.
10. Mo, Clinton, et al. "Keyframe extraction from motion capture sequences with graph based deep reinforcement learning." Proceedings of the 29th ACM International Conference on Multimedia. 2021.
11. Zarif, Sameh, et al. "A Deep Learning Pipeline for Adaptive Bayesian-Optimized Keyframe Extraction in Video Summarization." IEEE Access (2026).
12. Zheng, Bowen, et al. "Autokeyframe: Autoregressive keyframe generation for human motion synthesis and editing." Proceedings of the Special Interest Group on Computer Graphics and Interactive Techniques Conference Conference Papers. 2025.
13. Tan, Guanghua, and Jinxi Zou. "A Key Frame Extraction Method for Intelligent Patrol Video of Large-Scale Substation Based on Sequence Segmentation." Proceedings of the 2023 International Conference on Big Data Mining and Information Processing. 2023.
14. Kavitha, J., et al. "A dual-optimization strategy: de-noising and MCDM weighting for key frame extraction in deep learning." Expert Systems with Applications 290 (2025): 128351.
15. Ashiquzzaman, Akm, et al. "Video Key Frame Extraction & Fire-Smoke Detection with Deep Compact Convolutional Neural Network." The 9th International Conference on Smart Media and Applications. 2020.
16. Singh, Pratibha, and Alok Kumar Singh Kushwaha. "Key frame extraction algorithm for video summarization based on key frame extraction using sliding window." Multimedia Tools and Applications 84.26 (2025): 31793-31812.
17. Lv, Chaohui, and Yiyang Huang. "Effective keyframe extraction from personal video by using nearest neighbor clustering." 2018 11th International Congress on

Image and Signal Processing, BioMedical Engineering and Informatics (CISP-BMEI). IEEE, 2018.
18. Xiao, Yang Dong, Cheng Xin Min, and Ran Mao Ke. "Method of video key frame extraction based on lightweight-SAM." Proceedings of the 2024 7th International Conference on Big Data Technologies. 2024.
19. Ai, Yu, Ce Xu, and Yansong Bai. "Key frame extraction method based on video compressed domain feature fusion." Proceedings of the 2023 6th international conference on artificial intelligence and pattern recognition. 2023.
20. Navyasri, M., and G. Jaya Suma. "A novel key frame extraction using a deep learning model for sign language recognition on videos." Multimedia Tools and Applications (2025): 1-29.
21. Bhuyan, Himadri, et al. "An automated method for identification of key frames in Bharatanatyam dance videos." IEEE Access 9 (2021): 72670-72680.
22. Peer Mohamed Appa, M. A. Y., et al. "Key frame extraction based abnormal vehicle identification technique using statistical distribution analysis." Scientific Reports 15.1 (2025): 30957.
23. Qiao, Shunli, and Honglin Liu. "Automatic classification of criminal activities for security surveillance by keyframes detection and advanced inception techniques." Scientific Reports (2025).
24. He, Kaiming, et al. "Deep residual learning for image recognition." Proceedings of the IEEE conference on computer vision and pattern recognition. 2016.
25. Huang, Gao, et al. "Densely connected convolutional networks." Proceedings of the IEEE conference on computer vision and pattern recognition. 2017.
26. Tan, Mingxing, and Quoc Le. "Efficientnet: Rethinking model scaling for convolutional neural networks." International conference on machine learning. PMLR, 2019.
27. Sandler, Mark, et al. "Mobilenetv2: Inverted residuals and linear bottlenecks." Proceedings of the IEEE conference on computer vision and pattern recognition. 2018.
28. Radford, Alec, et al. "Improving language understanding by generative pre-training." (2018).
29. Bordes, Florian, et al. "An introduction to vision-language modeling." arXiv preprint arXiv:2405.17247 (2024).
30. Radford, Alec, et al. "Learning transferable visual models from natural language supervision." International conference on machine learning. PmLR, 2021.
31. Liu, Yiming, et al. "Data or Language Supervision: What Makes CLIP Better than DINO?." arXiv preprint arXiv:2510.11835 (2025).
32. Ghosh, Soumitra, Asif Ekbal, and Pushpak Bhattacharyya. "Natural language processing and sentiment analysis: perspectives from computational intelligence." Computational intelligence applications for text and sentiment data analysis. Academic Press, 2023. 17-47.